

Lab Report No 13
Image Transformation into Frequency Domain

Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

26th Jan, 2025

Department of Computer Engineering,
HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)

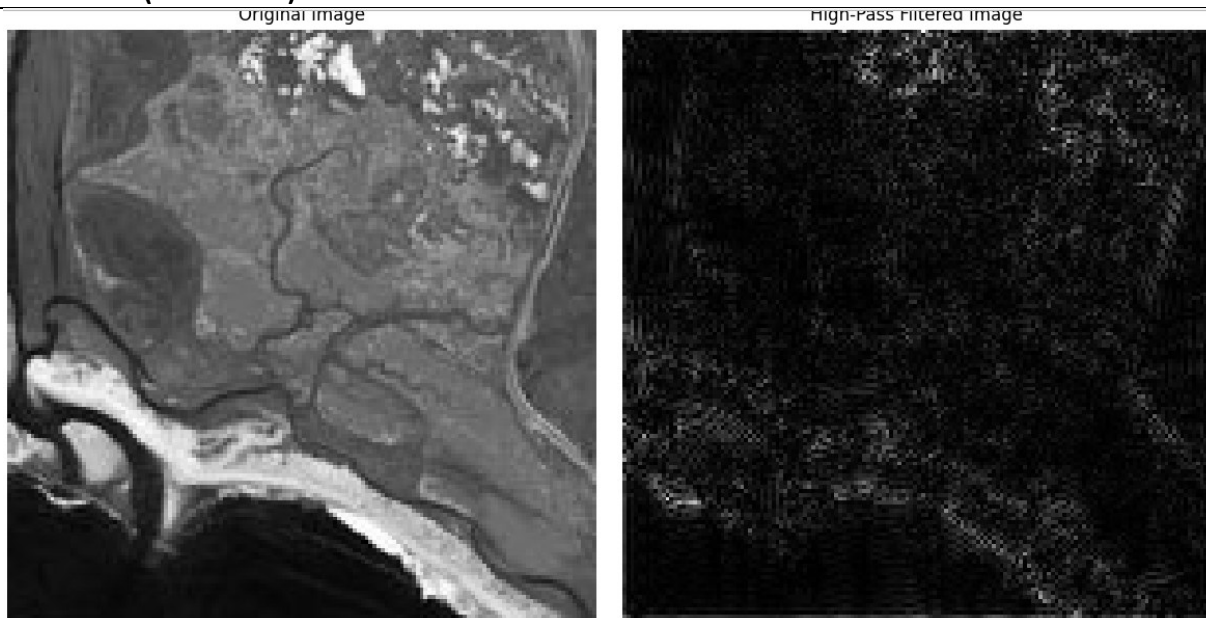
Perform the high-pass filtering of the frequency spectrum of an image.
--

The code

<pre>from PIL import Image import numpy as np import matplotlib.pyplot as plt import cv2 # Load the image image_path = "high pass filter.jpg" image = Image.open(image_path).convert("L") # Convert to grayscale # Convert the image to a NumPy array image_array = np.array(image) # Apply Fourier Transform to the image f = np.fft.fft2(image_array) fshift = np.fft.fftshift(f) # Create a High-Pass Filter (HPF) rows, cols = image_array.shape crow, ccol = rows // 2, cols // 2 # Create a mask with a centered square of zeros (low-pass block) mask = np.ones((rows, cols), np.uint8) size = 30 # Define the size of the low-pass block mask[crow - size:crow + size, ccol - size:ccol + size] = 0 # Apply the mask to the frequency domain fshift_filtered = fshift * mask # Inverse Fourier Transform to get the filtered image f_ishift = np.fft.ifftshift(fshift_filtered) image_filtered = np.abs(np.fft.ifft2(f_ishift)) # Plot the original and high-pass filtered images plt.figure(figsize=(12, 6)) plt.subplot(1, 2, 1) plt.title("Original Image")</pre>
--

```
plt.imshow(image_array, cmap="gray")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.title("High-Pass Filtered Image")
plt.imshow(image_filtered, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Task No 2:

Solution:

Brief description (3-5 lines)

It means producing an ideal low-pass filter and an ideal high-pass filter that can be chosen by the user. Compute the DFT of the image and multiply the transformed image with the filter. Inverse DFT will then be performed for the image and the result displayed.

The code

```
import numpy as np
```

```

import matplotlib.pyplot as plt
from PIL import Image
from scipy.fft import fft2, fftshift, ifft2
# Load and convert the image to grayscale
image = Image.open("ideal low
passandhighpassfilter.jpg").convert("L")
image_array = np.array(image)
# Function to create an ideal low-pass filter
def ideal_low_pass_filter(shape, radius):
    rows, cols = shape
    center_row, center_col = rows // 2, cols // 2
    mask = np.zeros((rows, cols), dtype=float)
    for i in range(rows):
        for j in range(cols):
            distance = np.sqrt((i - center_row) ** 2 + (j - center_col) ** 2)
            if distance <= radius:
                mask[i, j] = 1
    return mask
# Function to create an ideal high-pass filter
def ideal_high_pass_filter(shape, radius):
    return 1 - ideal_low_pass_filter(shape, radius)
# Apply filter in frequency domain
def apply_filter(image, filter_type, radius):
    # Compute DFT of the image
    dft = fft2(image)
    dft_shifted = fftshift(dft)
    # Create the filter
    if filter_type == "low":
        filter_mask = ideal_low_pass_filter(image.shape, radius)
    elif filter_type == "high":
        filter_mask = ideal_high_pass_filter(image.shape, radius)
    else:
        raise ValueError("filter_type must be 'low' or 'high'")
    # Apply the filter
    filtered_dft = dft_shifted * filter_mask

```

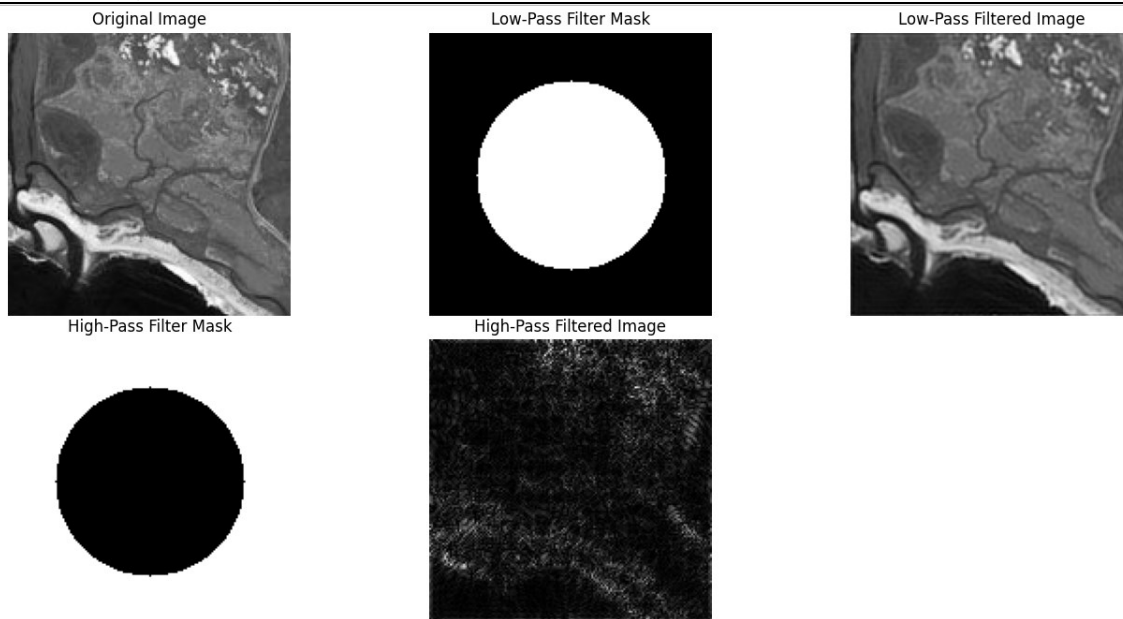
```

# Inverse DFT to reconstruct the image
filtered_dft_shifted = np.fft.ifftshift(filtered_dft)
filtered_image = np.abs(fft2(filtered_dft_shifted))
return filtered_image, filter_mask
# Parameters for filtering
filter_radius = 50 # Radius for the filter
# Apply low-pass filter
low_pass_image, low_pass_mask = apply_filter(image_array, "low",
filter_radius)
# Apply high-pass filter
high_pass_image, high_pass_mask = apply_filter(image_array, "high",
filter_radius)
# Plot results
plt.figure(figsize=(12, 8))
# Original image
plt.subplot(2, 3, 1)
plt.title("Original Image")
plt.imshow(image_array, cmap="gray")
plt.axis("off")
# Low-pass filter mask
plt.subplot(2, 3, 2)
plt.title("Low-Pass Filter Mask")
plt.imshow(low_pass_mask, cmap="gray")
plt.axis("off")
# Low-pass filtered image
plt.subplot(2, 3, 3)
plt.title("Low-Pass Filtered Image")
plt.imshow(low_pass_image, cmap="gray")
plt.axis("off")
# High-pass filter mask
plt.subplot(2, 3, 4)
plt.title("High-Pass Filter Mask")
plt.imshow(high_pass_mask, cmap="gray")
plt.axis("off")
# High-pass filtered image

```

```
plt.subplot(2, 3, 5)
plt.title("High-Pass Filtered Image")
plt.imshow(high_pass_image, cmap="gray")
plt.axis("off")
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Conclusion

In conclusion, the image transformation into frequency domains enables image analysis and processing according to frequency components and not just spatial details, techniques such as the Fourier Transform reveal crucial patterns for the efficient execution of subsequent tasks, such as filtering, compression, and noise reduction. This very approach is the foundation of various applications, including image enhancement, watermarking, and signal processing.